

WEEK-4 PRACTICAL

Task 1: Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

CODE:

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    int pid[3], status;

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n", getpid());

    for (int i = 0; i < 3; i++)
    {
        pid[i] = fork();

        if (pid[i] == 0)
        {
            printf("Child %d started. PID = %d, Parent PID = %d\n",
                i + 1, getpid(), getppid());
            sleep(i + 1);
            exit(10 + i);
        }
    }

    printf("All 3 child processes have been created.\n");

    printf("Using wait():\n");
    wait(&status);
    printf("wait() detected child PID finished with exit status %d\n",
        WEXITSTATUS(status));

    printf("Using waitpid():\n");

    for (int i = 0; i < 3; i++)
    {
        waitpid(pid[i], &status, 0);
        printf("waitpid() detected child PID %d finished with exit status %d\n",
            pid[i], WEXITSTATUS(status));
    }

    printf("Parent process completed.\n");

    return 0;
}
```

OUTPUT:

```
navaneeth@Navaneeth: ~  
navaneeth@Navaneeth:~$ nano ProcessParent.c  
navaneeth@Navaneeth:~$ gcc ProcessParent.c -o ProcessParent  
navaneeth@Navaneeth:~$ ./ProcessParent  
Parent Process Started  
Parent PID: 567  
Child 1 started. PID = 568, Parent PID = 567  
Child 2 started. PID = 569, Parent PID = 567  
All 3 child processes have been created.  
Using wait():  
Child 3 started. PID = 570, Parent PID = 567  
wait() detected child PID finished with exit status 10  
Using waitpid():  
waitpid() detected child PID 568 finished with exit status 10  
waitpid() detected child PID 569 finished with exit status 11  
waitpid() detected child PID 570 finished with exit status 12  
Parent process completed.  
navaneeth@Navaneeth:~$ |
```

REPORT: The Multiple child processes were created successfully and their completion was synchronized using wait() and waitpid().

Task 2: Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

CODE:

```
GNU nano 8.7.1 ChildProcess.c *  
#include <stdio.h>  
#include <unistd.h>  
#include <stdlib.h>  
  
int main()  
{  
    int pid = fork();  
  
    if (pid == 0)  
    {  
        printf("Child process: PID = %d\n", getpid());  
        printf("Child process terminating...\n");  
        exit(0);  
    }  
    else  
    {  
        printf("Parent process: PID = %d\n", getpid());  
        printf("Child PID = %d\n", pid);  
  
        sleep(10);  
  
        printf("Parent terminating...\n");  
    }  
  
    return 0;  
}
```

OUTPUT:

```
navaneeth@Navaneeth:~$ nano ChildProcess.c
navaneeth@Navaneeth:~$ gcc ChildProcess.c -o ChildProcess
navaneeth@Navaneeth:~$ ./ChildProcess
Parent process: PID = 671
Child PID = 672
Child process: PID = 672
Child process terminating...
Parent terminating...
navaneeth@Navaneeth:~$ |
```

REPORT: The zombie process was successfully observed, and using wait() in the parent process prevented the child from remaining as a zombie.

PS -EL:

```
navaneeth@Navaneeth:~$ ps -el
F S  UID      PID      PPID  C  PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
4 S   0         1         0  0  80   0 -  6078 -        ?    00:00:00 systemd
5 S   0         2         1  0  80   0 -   795 -      hvc0    00:00:00 init-systemd(Ub
0 S   0         8         2  0  80   0 -   795 -      hvc0    00:00:00 init
4 S   0        48         1  0  79  -1 - 12606 -        ?    00:00:00 systemd-journal
4 S  989        80         1  0  80   0 -   5635 -        ?    00:00:00 systemd-resolve
4 S   0        90         1  0  80   0 -   8811 -        ?    00:00:00 systemd-udev
4 S   0       133         1  0  80   0 -    722 -        ?    00:00:00 chronyd-starter
4 S   0       142         1  0  80   0 -   1118 -        ?    00:00:00 cron
4 S  995       149         1  0  80   0 -   2205 -        ?    00:00:00 dbus-daemon
4 S   0       154         1  0  80   0 -  11023 -        ?    00:00:00 networkd-dispat
4 S   0       158         1  0  80   0 -   4608 -        ?    00:00:00 systemd-logind
4 S  100       178         1  0  80   0 -  55137 -        ?    00:00:00 rsyslogd
4 S  988       218       133  0  80   0 -   5105 -        ?    00:00:00 chronyd
1 S  988       229       218  0  80   0 -   3023 -        ?    00:00:00 chronyd
4 S   0       271         1  0  80   0 -  30864 -        ?    00:00:00 unattended-upgr
4 S   0       273         1  0  80   0 -   1373 -      tty1    00:00:00agetty
5 S   0       326         2  0  80   0 -    796 -        ?    00:00:00 SessionLeader
5 S   0       327       326  0  80   0 -    800 -        ?    00:00:00 Relay(328)
4 S 1000       328       327  0  80   0 -   1567 do_wai pts/0    00:00:00 bash
4 S   0       329         2  0  80   0 -   2163 -        ?    00:00:00 login
4 S 1000       371         1  0  80   0 -   5688 -        ?    00:00:00 systemd
5 S 1000       375       371  0  80   0 -   5728 -        ?    00:00:00 (sd-pam)
4 S 1000       398       329  0  80   0 -   1534 poll_s pts/1    00:00:00 bash
0 R 1000       682       328  0  80   0 -   1798 -      pts/0    00:00:00 ps
navaneeth@Navaneeth:~$ |
```

REPORT: The process table was successfully observed using ps -el, and the zombie process was identified by the Z state.